

RMI (Remote Method Invocation) – Summary

Remote Method Invocation (RMI) is a Java technology that enables communication between Java applications running on different machines in a distributed environment. Using RMI, an object in one JVM can invoke methods of an object located in another JVM as if it were a local object. This helps in developing distributed client-server applications.

RMI architecture consists of client, server, stub, skeleton, and RMI registry. The RMI registry acts as a naming service that allows clients to locate remote objects available on the server.

Key Components of RMI

1. Remote Object

An object whose methods can be invoked remotely by another application.

2. Remote Interface

Defines methods available for remote access.

- Extends `java.rmi.Remote`
- Methods throw `RemoteException`

3. Stub

A client-side proxy object.

Responsibilities:

- Establish connection with remote JVM
- Marshal parameters
- Send request and receive response

4. Skeleton

A server-side gateway.

Responsibilities:

- **Receive request from stub**
- **Invoke actual method**
- **Return result to client**

5. RMI Registry

A naming service that binds remote objects to names so clients can locate them.

Working of RMI

- 1. Client requests remote method**
- 2. Stub receives request and sends it over network**
- 3. Skeleton on server receives request**
- 4. Actual remote method executes**
- 5. Result is sent back to client**

This creates transparent communication between distributed applications.

Implementation Steps

1. Create Remote Interface

Defines remote methods like `add()`.

2. Implement Remote Interface

Server class implements the interface and extends `UnicastRemoteObject`.

3. Create Server Program

- Creates remote object
- Registers it using `Naming.rebind()`

4. Create Client Program

- Uses `Naming.lookup()`
 - Invokes remote methods
-

Stub Generation

Generate stub using:

```
rmic AddServerImpl
```

This creates:

- `AddServerImpl_Stub.class`
-

Execution Steps

Start RMI Registry

```
start rmiregistry
```

Run Server

```
java AddServer
```

Run Client

```
java AddClient <serverIP> num1 num2
```

Example:

java AddClient 192.168.1.1 7 8

The server computes the addition and returns the result to the client.

Advantages of RMI

- **Supports distributed computing**
 - **Easy remote communication**
 - **Object-oriented approach**
 - **Simplifies client-server applications**
-

Applications of RMI

- **Distributed systems**
 - **Client-server applications**
 - **Enterprise systems**
 - **Remote database access**
-
-
-

Distributed Application using CORBA – Summary

CORBA (Common Object Request Broker Architecture) is a **vendor-independent distributed computing framework** developed by the Object Management Group (OMG). It enables communication between applications written in different programming languages and running on different platforms. This makes it highly useful for integrating heterogeneous systems in a distributed environment.

CORBA uses an **Object Request Broker (ORB)**, which acts as a middleware that allows clients to invoke methods on remote objects as if they were local. This provides **location transparency** and simplifies distributed communication.

Key Components of CORBA

1. ORB (Object Request Broker)

- Core component of CORBA
- Handles communication between client and server
- Locates objects and forwards requests

2. IDL (Interface Definition Language)

- Defines interfaces between client and server
- Language-independent
- Used to generate **stubs and skeletons**

3. Stub and Skeleton

- **Stub (Client-side proxy)** → Sends request to server
- **Skeleton (Server-side proxy)** → Receives request and calls method

These components ensure seamless communication in distributed systems.

Working of CORBA

1. Client requests a remote method
2. Stub converts request into ORB format
3. ORB forwards request using **IIOP protocol**
4. Skeleton receives request on server
5. Server executes method
6. Response is sent back via ORB

This process allows **remote method invocation (RMI-like behavior)** across different systems.

Java Support for CORBA

Java provides built-in support for CORBA using:

- **Java IDL** → API for CORBA integration
- **idlj compiler** → Converts IDL to Java code
- **ORBD** → Naming service daemon

Java + CORBA provides:

- Network transparency (CORBA)
 - Platform independence (Java)
-

Steps to Develop CORBA Application

1. Define Interface (IDL)

Write interface using IDL specifying methods and data types.

2. Generate Java Code

Use `idlj -fall file.idl` to generate:

- Interface
- Stub
- Skeleton
- Helper & Holder classes

3. Implement Server

- Extend generated skeleton class
- Implement methods
- Register object with naming service

4. Implement Client

- Initialize ORB
 - Get object reference from naming service
 - Call remote methods
-

Execution Steps

1. Compile IDL → generate classes
2. Compile Java files
3. Start naming service (`orbd`)
4. Run server
5. Run client

The client invokes remote methods and receives results from server.

Example: Hello World CORBA

- Server implements `sayHello()` method
- Client calls method remotely
- Output: “**Hello World!**”

This demonstrates remote communication using CORBA architecture.

Advantages of CORBA

- Platform and language independent
 - Supports distributed systems
 - Promotes interoperability
 - Scalable and flexible architecture
-
-

Distributed System using MPI (MPJ Express) – Summary

Message Passing Interface (MPI) is a widely used **parallel programming model** that enables communication between multiple processes in a distributed system. It follows a **message-passing mechanism**, where processes exchange data to perform computations collaboratively. In Java, MPI is implemented using libraries like **MPJ Express**, which allows execution of parallel programs on multicore systems and clusters.

MPI follows a **Single Program Multiple Data (SPMD)** model, where multiple processes execute the same program but operate on different portions of data. This improves performance and efficiency for computational tasks like summation of large arrays.

Basic Concepts of MPI

1. Initialization & Finalization

- `MPI_Init()` → Starts MPI environment
- `MPI_Finalize()` → Ends execution

2. Communicator

- Group of processes that communicate with each other

- Default communicator: **MPI_COMM_WORLD**

3. Rank and Size

- **Rank** → Unique ID of each process
- **Size** → Total number of processes

These are essential for identifying and managing processes.

Message Passing Operations

1. Send Operation

Used to send data from one process to another:

`MPI_Send()`

2. Receive Operation

Used to receive data from another process:

`MPI_Recv()`

These operations enable communication between distributed processes.

Collective Communication

MPI supports group communication operations:

- **Scatter** → Divides data into parts and distributes to processes
- **Gather** → Collects results from all processes

In this assignment, the array is divided among processes using **scatter**, and partial sums are combined using **gather**.

Working of the Program

1. Input array of size **N**
2. Divide array into **N/n parts**
3. Distribute parts to **n processors**
4. Each processor calculates **partial sum**
5. Intermediate sums are displayed

6. Final sum is computed by combining results

This demonstrates parallel computation using MPI.

MPJ Express

MPJ Express is a **Java-based MPI library** used for:

- Parallel programming
- Multicore execution
- Cluster computing

It acts as middleware to manage communication between processes and supports scalable applications.

Implementation Steps

1. Install MPJ Express and set environment variables
 2. Write Java program (e.g., `ScatterGather.java`)
 3. Compile using:

```
javac -cp mpj.jar filename.java
```
 4. Execute using:

```
mpjrun.sh -np <no_of_processes> filename
```
-

Advantages of MPI

- Efficient parallel processing
 - Scalable to large systems
 - Supports distributed and shared memory
 - Improves performance of computation-heavy tasks
-
-

Berkeley Algorithm – Clock Synchronization in Distributed Systems (Summary)

A **distributed system** consists of multiple computers connected through a network that coordinate their actions using message passing. Since each system has its own internal clock, differences in time (clock drift) can occur, leading to inconsistencies. To maintain coordination and proper execution, **clock synchronization** is essential.

Clocks in distributed systems are synchronized using **physical clocks** based on **UTC (Coordinated Universal Time)** as a reference. Synchronization ensures all nodes maintain a consistent notion of time.

Types of Clock Synchronization

1. External Synchronization

- Uses an external reference clock (like UTC)
- Nodes adjust their clocks according to the reference

2. Internal Synchronization

- Nodes synchronize with each other
 - Adjust clocks based on mutual agreement
-

Clock Synchronization Algorithms

Centralized Algorithms

- Use a single time server
- Example: **Berkeley Algorithm**
- Limitation: Single point of failure

Distributed Algorithms

- No central server
 - Nodes coordinate among themselves
 - Example: NTP, averaging algorithms
-

Berkeley Algorithm Overview

The **Berkeley Algorithm** is a **centralized clock synchronization technique** where one node acts as a **master**, and others act as **slave nodes**. It is used when systems do not have access to an accurate external time source.

Working of Berkeley Algorithm

5

1. **Master Selection**
 - One node is selected as the master using a leader election algorithm
2. **Polling Slave Nodes**
 - Master sends requests to all slave nodes
 - Each node replies with its local time
3. **Time Difference Calculation**
 - Master calculates time differences between its clock and each slave clock
4. **Average Time Computation**
 - Master computes the average of all time differences
 - Ignores faulty or extreme values (outliers)
5. **Broadcast Adjustment**
 - Master sends adjustment values (not exact time) to all nodes
 - Each node updates its clock accordingly

Key Features

- Does not rely on external UTC source
- Reduces clock drift among nodes
- Improves consistency in distributed systems
- Uses **average time correction**

Limitations

- Single point of failure (master node)
- Communication delay affects accuracy
- Requires re-election if master fails

Scope of Improvements

- Ignore outliers in time calculations

- Use backup master node
 - Send relative time adjustments instead of exact time
 - Reduce latency in communication
-

Implementation (Python)

- Master acts as **clock server**
- Clients send their local time
- Master calculates average and sends correction
- Clients update their clocks

This demonstrates practical implementation of synchronization in distributed systems.

--

Token Ring Mutual Exclusion Algorithm – Summary

In distributed systems, **process synchronization** ensures that multiple processes coordinate properly while accessing shared resources. When a resource must be accessed by only one process at a time, the concept of **mutual exclusion** is used. It guarantees that no two processes enter the critical section simultaneously.

A good mutual exclusion algorithm must satisfy:

- **Safety** → Only one process accesses resource at a time
 - **Liveness** → Processes eventually get access
 - **Fairness** → No starvation; equal opportunity for all
-

Types of Mutual Exclusion Algorithms

1. **Centralized Algorithm** → Uses a coordinator
2. **Token-Based Algorithm** → Uses a circulating token
3. **Contention-Based Algorithm** → Resolves conflicts dynamically

The **Token Ring Algorithm** belongs to the token-based category.

Token Ring Algorithm Overview

In this algorithm, processes are arranged in a **logical ring structure**. A special message called a **token** is passed from one process to another in a fixed order.

- Only the process holding the token can enter the **critical section**
 - After completing its task, the process passes the token to the next process
-

Working of Token Ring Algorithm

1. **Initialization**
 - Token is initially given to process 0
 2. **Token Passing**
 - Token moves in ring order:
Process $i \rightarrow \text{Process } (i+1) \bmod n$
 3. **Critical Section Access**
 - If a process needs resource $\rightarrow$ it uses token
 - Executes critical section
 4. **Release & Forward Token**
 - After execution, process passes token to next
 5. **Idle Case**
 - If process doesn't need resource $\rightarrow$ directly passes token
-

Key Characteristics

- Ensures **mutual exclusion** (only one token exists)
 - Provides **fair access** (round-robin order)
 - No need for a central coordinator
 - Simple and easy to implement
-

Advantages

- No starvation due to fixed order
 - Predictable and fair scheduling
 - Fully distributed (no single point of failure like centralized system)
-

Disadvantages

- **Token loss problem** → system halts if token is lost
- Detecting lost token is difficult
- Delay increases with large number of processes
- Not efficient if many processes don't need resource

Comparison with Centralized Approach

Feature	Token Ring	Centralized
Control	Distributed	Single coordinator
Failure	Token loss issue	Single point failure
Fairness	High	Queue-based
Complexity	Moderate	Simple

Leader Election Algorithms – Summary

In distributed systems, a **leader (coordinator)** is required to manage tasks such as resource allocation, synchronization, and communication. However, if the coordinator fails, the system must **elect a new leader dynamically** using election algorithms.

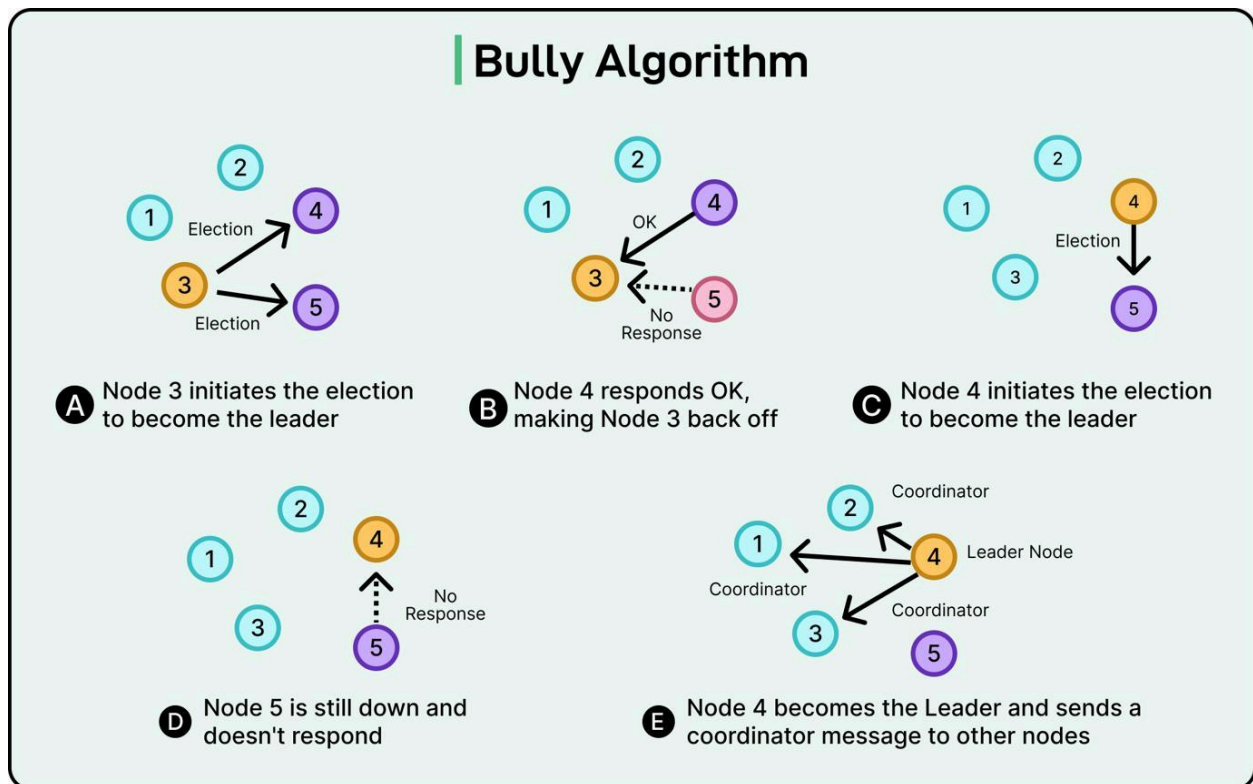
Each process in the system has a **unique ID**, and processes are aware of other process IDs. Based on system structure, two common election algorithms are used:

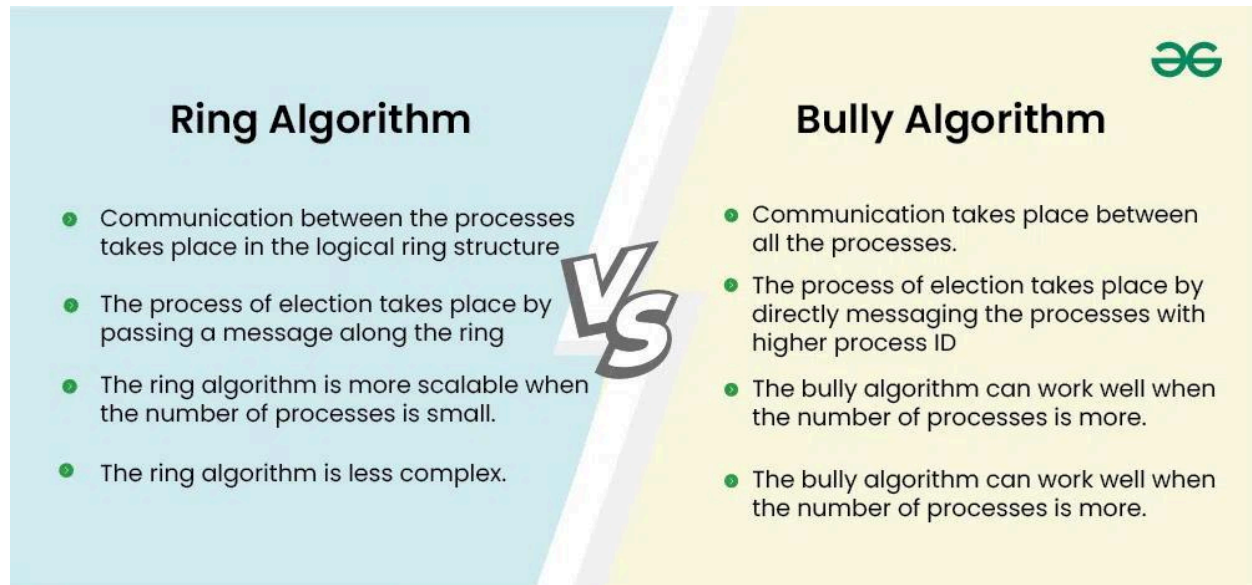
- **Bully Algorithm**
- **Ring Algorithm**

Bully Algorithm

The Bully Algorithm selects the process with the **highest ID** as the coordinator.

Working





1. A process detects that the coordinator has failed
2. It sends **ELECTION messages** to all higher-ID processes
3. If no one responds → it becomes the coordinator
4. If a higher process responds → that process takes over election
5. Winner broadcasts **COORDINATOR message** to all

Key Features

- Highest ID process always wins
- Fast election process
- Suitable when all processes can communicate directly

Advantages

- Simple and efficient
- Quick recovery from failure
- Ensures strongest (highest ID) process leads

Disadvantages

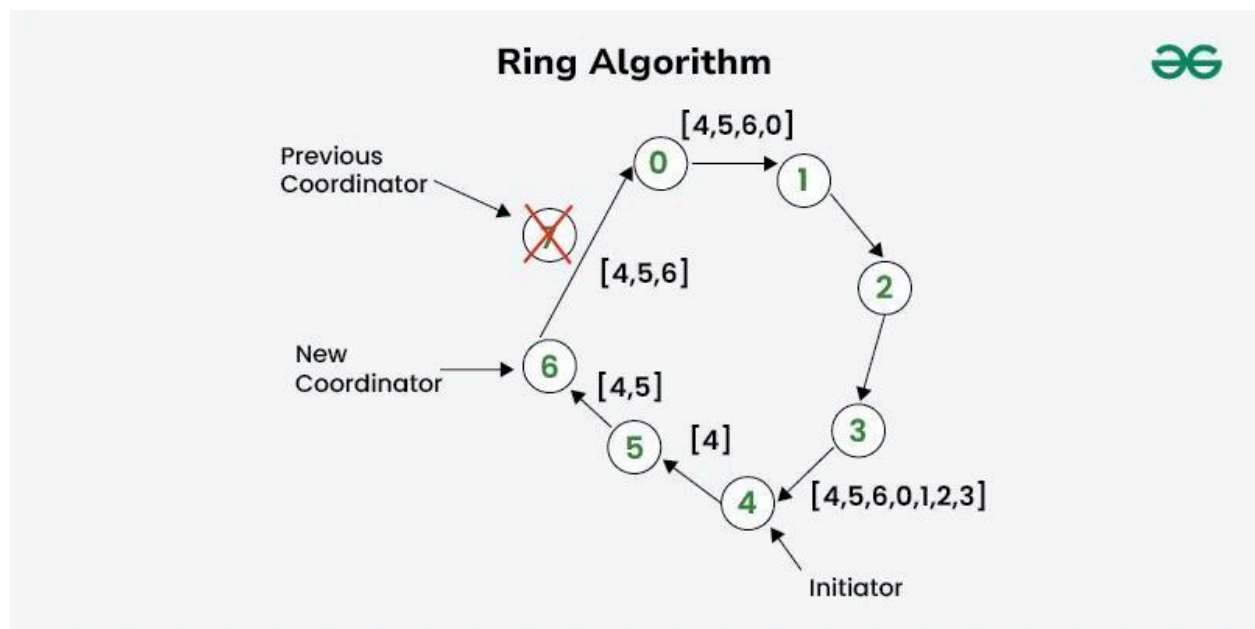
- High message overhead

- Not suitable for large systems
- Assumes knowledge of all processes

Ring Algorithm

The Ring Algorithm works in systems where processes are arranged in a **logical ring**.

Working



1. A process detects coordinator failure
2. It sends an **ELECTION message** to next active process
3. Each process adds its ID to the message
4. Message circulates the ring
5. Initiator selects process with **highest ID**
6. Coordinator is announced to all

Key Features

- Works in ring topology
 - Uses message passing in sequence
 - No central control required
-

Advantages

- Less message overhead than Bully
 - Fair and structured
 - Works well in large systems
-

Disadvantages

- Slower than Bully algorithm
 - Depends on ring structure
 - Failure handling can be complex
-

Comparison of Bully vs Ring Algorithm

Feature	Bully Algorithm	Ring Algorithm
Structure	Fully connected	Ring topology
Winner	Highest ID	Highest ID
Speed	Faster	Slower
Messages	High overhead	Moderate
Complexity	Simple	Moderate

Web Services & Distributed Application – Summary

A **web service** is a software system designed to enable communication between different applications over a network using standard protocols. It allows systems developed in different languages and platforms to interact seamlessly.

A service is considered a web service if it:

- Is **discoverable**
 - Uses **standard messaging format (XML/JSON)**
 - Is accessible over **internet/intranet**
 - Is **platform and language independent**
-

Types of Web Services

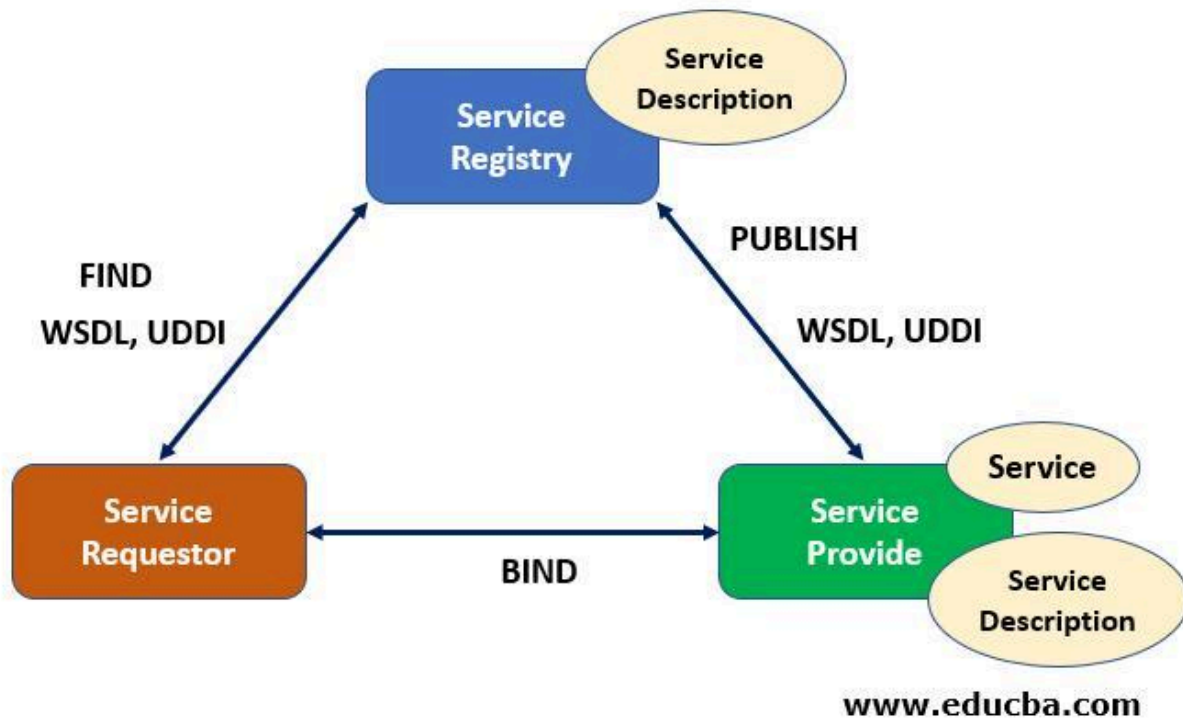
1. SOAP (Simple Object Access Protocol)

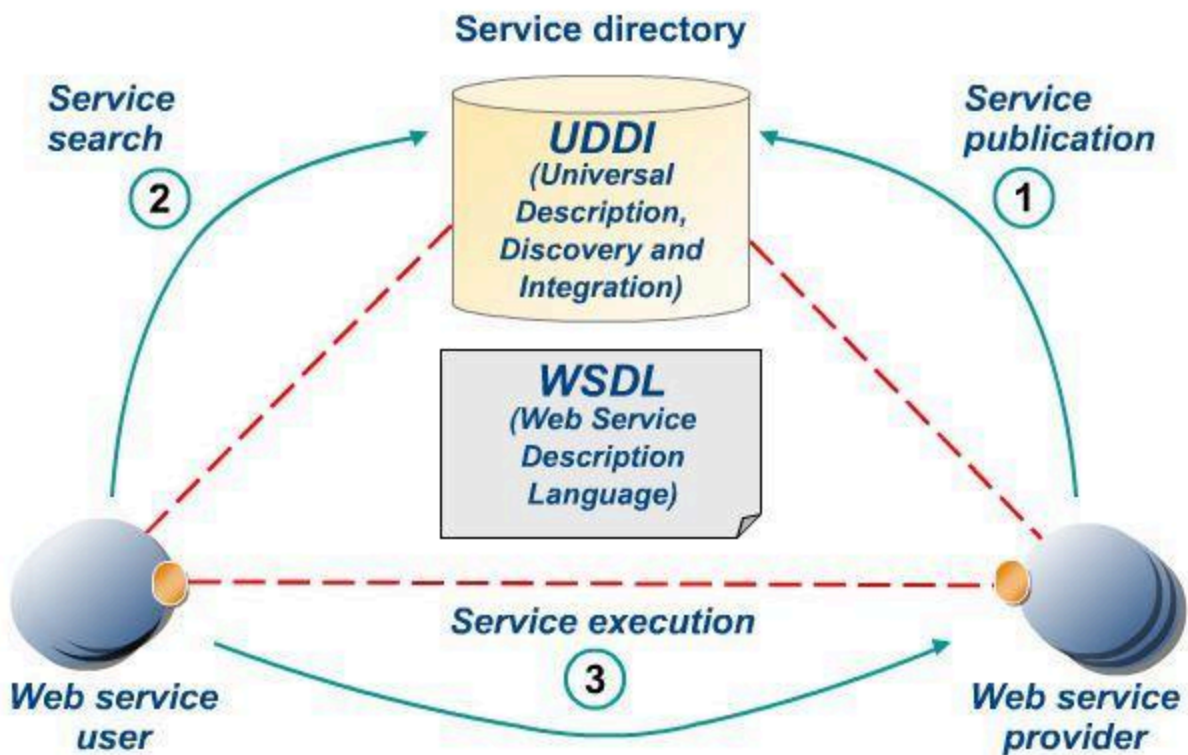
- XML-based protocol
- Platform independent
- Uses **WSDL** for service description
- More secure but complex

2. REST (Representational State Transfer)

- Architectural style (not protocol)
- Uses HTTP methods (GET, POST, PUT, DELETE)
- Lightweight and faster than SOAP
- Uses URIs to identify resources

Web Service Architecture





There are three main roles:

1. **Service Provider**
 - Creates and publishes web service
2. **Service Requestor (Client)**
 - Consumes and interacts with service
3. **Service Registry (UDDI)**
 - Stores service descriptions

Working of SOAP Web Service

1. Client prepares request in **SOAP (XML)** format
2. Sends request via HTTP POST
3. Web service processes request
4. Server generates response
5. Response sent back as SOAP message
6. Client reads and processes response

SOAP ensures structured and standardized communication.

Key Components of SOAP

- **WSDL (Web Services Description Language)** → Describes service interface
 - **UDDI (Universal Description, Discovery, Integration)** → Service registry
 - **XML** → Message format
-

RESTful Web Services

REST uses simple HTTP methods and is more efficient:

- GET → Retrieve data
- POST → Create data
- PUT → Update data
- DELETE → Remove data

REST is widely used due to simplicity and performance.

Java APIs for Web Services

1. **JAX-WS (SOAP)**
 - Used to create SOAP-based services
 2. **JAX-RS (REST)**
 - Used to create RESTful services
 - Uses annotations for easy development
-

Implementation Steps

1. Create Web Service

- Create project (CalculatorWSApplication)
- Define service class (CalculatorWS)
- Deploy using GlassFish server

2. Consume Web Service

- Create client project
- Generate client classes
- Call web service methods

3. Create Web Interface

- Use JSP/Servlet
 - Provide UI for user interaction
-

Advantages of Web Services

- Platform independent
 - Enables distributed computing
 - Promotes interoperability
 - Scalable and reusable
-
-
-

VIVA

Assignment 1 – RMI Viva Questions & Answers

(Implement multi-threaded client/server process communication using RMI)

1. What is RMI?

RMI (Remote Method Invocation) is a Java technology that allows a Java object running on one machine to invoke methods on an object running on another machine.

2. Why is RMI used?

RMI is used to build distributed applications where objects communicate remotely over a network.

3. What is a remote object?

A remote object is an object whose methods can be accessed from another JVM over a network.

4. What is a remote interface?

A remote interface defines methods that can be called remotely. It extends the `Remote` interface.

5. Why do remote methods throw `RemoteException`?

Because network communication may fail during remote calls.

6. What is a stub?

A stub is a client-side proxy object that forwards requests to the remote server object.

7. What is a skeleton?

A skeleton is a server-side object that receives requests from the stub and invokes actual methods.

8. What is the role of `rmiregistry`?

It stores remote object references and helps clients locate remote objects.

9. What is marshalling?

Marshalling means converting objects/data into a stream for network transmission.

10. What is unmarshalling?

It is the process of reconstructing transmitted data back into objects.

11. Which package is used for RMI?

```
java.rmi
```

12. Why does remote object extend `UnicastRemoteObject`?

To make the object available for remote communication.

13. What is `Naming.rebind()`?

It binds or registers a remote object with the RMI registry.

14. Difference between local and remote method invocation?

Local invocation occurs within same JVM, remote invocation occurs across different JVMs/machines.

15. What is serialization in RMI?

It converts objects into byte streams for transmission.

16. What files are required in RMI?

- Remote Interface
- Implementation class
- Server class
- Client class

17. Command to generate stub?

```
rmic AddServerImpl
```

18. Command to start registry?

19. Advantages of RMI?

- Easy distributed programming
- Object-oriented communication
- Platform independence

20. Limitations of RMI?

- Java dependent
- Slower due to network overhead

Assignment 2 – CORBA Viva Questions & Answers

(Distributed application using CORBA)

1. What is CORBA?

CORBA stands for Common Object Request Broker Architecture used for communication between distributed objects.

2. Who developed CORBA?

OMG (Object Management Group).

3. What is ORB?

ORB (Object Request Broker) handles communication between client and server objects.

4. What is IDL?

IDL (Interface Definition Language) defines interfaces for distributed objects.

5. Why is IDL important?

It allows communication between applications written in different languages.

6. What is IIOP?

Internet Inter-ORB Protocol used for communication between ORBs.

7. What is a stub in CORBA?

Client-side proxy object.

8. What is a skeleton in CORBA?

Server-side object that receives requests.

9. What is POA?

Portable Object Adapter used to connect object implementation with ORB.

10. What is naming service?

It maps object names to object references.

11. Command to generate Java files from IDL?

```
idlj -fall Hello.idl
```

12. What is orbd?

ORB daemon process providing naming service.

13. Difference between RMI and CORBA?

- RMI is Java-specific
- CORBA supports multiple languages

14. What are helper classes?

Classes used for narrowing and stream operations.

15. What are holder classes?

Used for `out` and `inout` parameters.

16. Advantages of CORBA?

- Language independent
- Platform independent
- Supports distributed computing

17. What is object reference?

Unique identifier used to access CORBA objects.

18. What is static invocation?

Invocation using pre-generated stubs and skeletons.

19. What is distributed object computing?

Objects communicate over network transparently.

20. Why is CORBA used?

To integrate heterogeneous distributed systems.

Assignment 3 – MPI / MPJ Express Viva Questions & Answers

(Distributed sum using MPI/OpenMP)

1. What is MPI?

MPI stands for Message Passing Interface used for parallel computing.

2. What is MPJ Express?

Java implementation of MPI.

3. What is parallel computing?

Multiple processors execute tasks simultaneously.

4. What is communicator?

Group of processes communicating together.

5. What is `MPI_COMM_WORLD`?

Default communicator containing all processes.

6. What is rank?

Unique ID assigned to each process.

7. What is process size?

Total number of processes in communicator.

8. Function to get process rank?

```
MPI_Comm_rank()
```

9. Function to get total processes?

```
MPI_Comm_size()
```

10. What is scatter operation?

Distributes data among processes.

11. What is gather operation?

Collects data from processes.

12. What is SPMD?

Single Program Multiple Data model.

13. What is multicore configuration?

Execution on multiple cores of same machine.

14. What is message passing?

Processes communicate by sending messages.

15. Why MPI is used?

For high-performance parallel processing.

16. What is synchronization in MPI?

Coordination among parallel processes.

17. What is `MPI_Send`?

Function to send messages.

18. What is `MPI_Recv`?

Function to receive messages.

19. Advantages of MPJ Express?

- Portable
- Scalable
- Efficient parallel execution

20. Command to run MPJ program?

```
mpjrun.sh -np 4 Assign2
```

Assignment 4 – Berkeley Algorithm Viva Questions & Answers

(Clock Synchronization)

1. What is clock synchronization?

Adjusting clocks of distributed systems to same time.

2. What is Berkeley Algorithm?

Centralized clock synchronization algorithm.

3. Who acts as master in Berkeley Algorithm?

One selected node acts as master.

4. What do slave nodes do?

Send their local times to master.

5. How does master synchronize clocks?

Calculates average time difference and broadcasts corrected time.

6. Is Berkeley algorithm centralized?

Yes.

7. What is UTC?

Universal Coordinated Time.

8. Difference between internal and external synchronization?

- Internal: synchronize within system
- External: synchronize with UTC reference

9. Advantage of Berkeley algorithm?

No need of external UTC clock.

10. Limitation of Berkeley algorithm?

Master node failure affects synchronization.

11. What is Cristian's Algorithm?

Method for fetching client clock times.

12. What is latency?

Delay in communication.

13. What are outliers?

Abnormal clock values deviating from average.

14. Why synchronization is needed?

To coordinate distributed processes.

15. What happens if clocks differ?

Inconsistency in distributed operations.

16. What language was used in assignment?

Python.

17. What is socket programming?

Communication between systems using sockets.

18. What is master-slave architecture?

One master controls multiple slave nodes.

19. Applications of synchronization?

Distributed databases, banking, cloud systems.

20. Main conclusion of Berkeley algorithm?

All systems maintain approximately same time.

Assignment 5 – Token Ring Mutual Exclusion Viva Questions & Answers

1. What is mutual exclusion?

Ensuring only one process accesses critical section at a time.

2. What is Token Ring Algorithm?

Distributed mutual exclusion algorithm using a token.

3. What is a token?

Special permission message allowing access to critical section.

4. What is ring topology?

Processes connected in circular manner.

5. Which process enters critical section?

Only process holding token.

6. What happens after execution?

Token passed to next process.

7. What is critical section?

Code accessing shared resource.

8. Advantages of token ring?

- Fairness
- No starvation

9. Disadvantage of token ring?

Token loss causes failure.

10. What is starvation?

Process waits indefinitely.

11. What is fairness?

Every process gets equal chance.

12. What is liveness?

System continues making progress.

13. What is safety?

Only one process enters critical section.

14. Why synchronization needed?

Avoid data inconsistency.

15. What if token is duplicated?

Multiple processes may enter critical section.

16. How token loss handled?

Generate new token using recovery algorithm.

17. Difference between centralized and distributed mutual exclusion?

Centralized uses coordinator, distributed does not.

18. What is deadlock?

Processes wait forever for resources.

19. Real-life example?

Single printer shared among users.

20. Main benefit of token ring?

Guaranteed mutual exclusion with orderly access.